

1. A linear queue with an array of size 8. Perform the operations: enqueue(10), enqueue(20), enqueue(30), dequeue(),dequeue(), enqueue(34), enqueue(50), enqueue(65)

a. What is the value of front and rear after each operation.

b. Will overflow occur? Explain.

Show final queue and the values.

2. Max=100; //maximum queue size 0 to 99

enqueue(int val); //insert in queue

int dequeue(); //remove from queue

int start; // start of queue.

int end; //end of queue

Based on the above pseudocode, answer the following:

- i) For circular queue, if the start =50 end =99,
what will be the new start and end after 2 enqueue () and 2 dequeue()
operations?

3. Abstract Data Type. Why stack is abstract data type. Why abstract data type used?

4. Convert the following infix to postfix expression using stack.

K+L-M*N + (O^P) * W/U/V *T+Q

$$K+L-M*N+(O^P)*W/U/V*T+Q$$

Step	Symbol	Stack	Output	Action
1	K	(empty)	K	Operand $\rightarrow$ output
2	+	+	K	Stack empty $\rightarrow$ push
3	L	+	K L	Operand $\rightarrow$ output
4	-	-	K L +	Pop + (same precedence) $\rightarrow$ output, push -
5	M	-	K L + M	Operand $\rightarrow$ output
6	*	- *	K L + M	Higher precedence than - $\rightarrow$ push *
7	N	- *	K L + M N	Operand $\rightarrow$ output
8	+	+	K L + M N * -	Pop * $\rightarrow$ output, pop - $\rightarrow$ output, push +
9	(	+(	K L + M N * -	Push (
10	O	+(	K L + M N * - O	Operand $\rightarrow$ output
11	^	+(^	K L + M N * - O	Push ^
12	P	+(^	K L + M N * - O P	Operand $\rightarrow$ output
13	)	+	K L + M N * - O P ^	Pop ^ $\rightarrow$ output, pop (
14	*	+ *	K L + M N * - O P ^	Higher precedence than + $\rightarrow$ push *
15	W	+ *	K L + M N * - O P ^ W	Operand $\rightarrow$ output
16	/	+ /	K L + M N * - O P ^ W *	Pop * $\rightarrow$ output, push /
17	U	+ /	K L + M N * - O P ^ W * U	Operand $\rightarrow$ output
18	/	+ /	K L + M N * - O P ^ W * U /	Pop / $\rightarrow$ output, push /
19	V	+ /	K L + M N * - O P ^ W * U / V	Operand $\rightarrow$ output
20	*	+ *	K L + M N * - O P ^ W * U / V /	Pop / $\rightarrow$ output, push *
21	T	+ *	K L + M N * - O P ^ W * U / V / T	Operand $\rightarrow$ output
22	+	+	K L + M N * - O P ^ W * U / V / T * +	Pop * $\rightarrow$ output, pop + $\rightarrow$ output, push +
23	Q	+	K L + M N * - O P ^ W * U / V / T * + Q	Operand $\rightarrow$ output
24	End	(empty)	K L + M N * - O P ^ W * U / V / T * + Q +	Pop remaining + $\rightarrow$ output

```

5. for(i=2; i<=n; i=i*2)
   for(j=1; j<=n; j++)
     sum++;

```

The time and space complexities of the above C code -----

Answer

Outer Loop (i loop):

- i starts at 1 and multiplies by 2 each time: $i = 1, 2, 4, 8, \dots, n$
- Number of iterations: $\approx O(\log n)$

Inner Loop (j loop):

- j runs from 1 to n each time: **n iterations**

Total iterations:

Total=(outer iterations)×(inner iterations)= $O(\log n) \times O(n) = O(n \log n)$

6. Consider the following sequence of operations on an empty stack:

```

push(54); push(52); pop(); push(55); push(62);
s = pop();

```

Consider the following sequence of operations on an empty queue: enqueue(21); enqueue(24); dequeue(); enqueue(28); enqueue(32); Q=dequeue();

The value of s+q is -----

7. The overflow condition in circular queue is represented by

- if((rear+1)%maxsize)==front)
- if((rear+1)%maxsize)=front)
- if((rear)%maxsize)-front)
- None of these

8. Which of the following is a valid node creation of doubly linked list?

- ```

struct node{
 struct node prev;
 int data;
 struct node next;
};

```
- ```

struct node* {
struct node * prev;
int* data;
struct node * next;
};

```

9. Why would you use a doubly linked list instead of a single linked list? What are the disadvantages of a doubly linked list over a singly linked list?

10. Time and space complexity

(i) Algorithm Factorial1(n)

```
factorial = 1
for i = 1 to n
do
factorial = factorial × i
return factorial
```

(ii) Algorithm Factorial2(n)

```
if n = 0 then
return 1
else
return n × Factorial2(n - 1)
```

Space Complexity

Because this is **recursive**, each function call is stored in the **call stack**.

If $n = 4$, the stack will contain:

```
Factorial2(4)
Factorial2(3)
Factorial2(2)
Factorial2(1)
Factorial2(0)
```

So there are $n + 1$ stack frames.

Space Complexity = $O(n)$

What is the time complexity of a push operation in a stack and to get from the stack, respectively?

- a. $O(1)$ and $O(n)$
- b. $O(\log n)$ and $O(n)$
- c. $O(n)$ and $O(1)$

1. Time Complexity

- **Step 1:** `factorial = 1` $\rightarrow O(1)$
- **Step 2:** `for i = 1 to n` $\rightarrow$ loop runs **n** times
 - Inside the loop: `factorial = factorial × i` $\rightarrow O(1)$ operation per iteration
- **Total time:**
 $O(1) + n \times O(1) = O(n)$
-

So, the **time complexity** is **$O(n)$** .

Space Complexity

- The algorithm uses:
 - 1 variable for `factorial`
 - 1 loop variable `i`
- **No recursion or extra data structures are used**, so space does **not** grow with **n**.

✓ Therefore, the **space complexity** is **$O(1)$**

Factorial2(4)

$\rightarrow 4 \times \text{Factorial2}(3)$

$\rightarrow 4 \times 3 \times \text{Factorial2}(2)$

$\rightarrow 4 \times 3 \times 2 \times \text{Factorial2}(1)$

$\rightarrow 4 \times 3 \times 2 \times 1 \times \text{Factorial2}(0)$

$\rightarrow 4 \times 3 \times 2 \times 1 \times 1$

The function calls itself **n times** until it reaches 0.

$T(n) = T(n-1) + O(1)$

Time Complexity = $O(n)$

d. $O(1)$ and $O(n \log n)$

Explanation:

- **Push operation in a stack $\rightarrow O(1)$**
Because insertion happens at the top of the stack without shifting elements.
- **Getting the minimum element (in a normal stack) $\rightarrow O(n)$**
Since you must traverse all elements to find the smallest one.

11. Post fix evaluation

5 6 7 8 + - * 4 +

Initially stack is empty

Token	Action	Stack
5	Operand $\rightarrow$ push 5	[5]
6	Operand $\rightarrow$ push 6	[5, 6]
7	Operand $\rightarrow$ push 7	[5, 6, 7]
8	Operand $\rightarrow$ push 8	[5, 6, 7, 8]
+	Pop 8 & 7 $\rightarrow 7+8=15$, push 15	[5, 6, 15]
-	Pop 15 & 6 $\rightarrow 6-15=-9$, push -9	[5, -9]
*	Pop -9 & 5 $\rightarrow 5 * -9 = -45$, push -45	[-45]
4	Operand $\rightarrow$ push 4	[-45, 4]
+	Pop 4 & -45 $\rightarrow -45 + 4 = -41$, push -41	[-41]

Infix Notation

A+B*(C+D)

Postfix Notation

A B C D + * +

Symbol	Stack	output	
A		A	
+	+	A	
B	+	AB	
*	+	AB	
(	+(	AB	
C	+(	ABC	
+	+(+	ABC	
D	+(+	ABCD	
)	+	ABCD+	
		ABCD+*	

Symbol	Stack	output	
(	(		
D	(	D	
+	(+	D	
C		DC+	
)		DC+	

Prefix Notation

**Reverse scan $A+B*(C+D)$
 $(D+C)*B+A$**

**Now reverse Postfix
 $DC+B*A+$**

Prefix

$+ A * B + C D$

12. Derive the condition for queue full in a circular queue with size 5 implemented using array.

Perform enqueue(10), enqueue(20), enqueue(36), enqueue(48), dequeue(), enqueue(5), enqueue(6).